Soapy GrooveGraph Technical Reference

Matt Kennedy

Section 1: System Overview

Soapy GrooveGraph (SGG) is an AI music discovery tool built over a personal music library. The SGG extracts audio features from raw music files, stores them as vectors in a vector database, and answers natural language queries via a retrieval-augmented generation (RAG) pipeline. All computation runs locally — no cloud services, no external APIs for core functionality.

Purpose: enable natural language discovery of music within a personal library. Users ask questions, or make selections, and AI will provide ranked track results through an interactive dashboard.

In-Short: SGG knows *your* library. Every obscure album you imported from a CD in 2003. Every artist no one else is familiar with. SGG lets you ask questions about your own music library and finds unique tracks and provides recommendations that were forgotten or buried in your collection.

Design Principles

- **Local-first** — all models, databases, and compute run on personal computer
- **Two data sources** — The main data source is Essentia audio features extracted from personal iTunes library files. A second source was needed for missing album covers using the iTunes API.
- **Distinct layers** — feature extraction, data transformation, vector storage, and UI are discrete pipeline stages
- **Identifier consistency** — `file_hash` is the primary key carried through every stage of the pipeline

Tool Stack

Layer	Tool	Purpose
Audio extraction	Essentia (Docker)	Extract audio features and metadata from iTunes audio files
Data transformation	dbt-core + DuckDB	Stage, deduplicate, and build the audio feature mart
Vector store	Qdrant	Store and query audio and text embedding vectors
Embeddings	Ollama — `nomic-embed-text`	Generate 768-dim text embeddings from track descriptions
LLM	Ollama — `gemma3:27b`	Generate cited natural language answers from retrieved context
Album art	iTunes Search API (cached)	Resolve album artwork URLs — cached locally, never called at query time
UI	Streamlit	Interactive dashboard — RAG query, audio similarity, feature inspector

Section 2: Architecture & Data Flow

SGG is a linear, stage-gated pipeline. Each stage produces a discrete output consumed by the next. The pipeline is divided into two phases: **setup** (Stages 1–5, run once) and **runtime** (Stage 6, runs on every user query).

Pipeline Diagram

```
iTunes Audio Library (~9,254 files)
        │
        ▼
Stage 1 — File Manifest
        scripts/es_select_files.py → data/file_lists/es_targets.txt
        │
        ▼
Stage 2 — Audio Feature Extraction
        Essentia (Docker, temporary) → data/raw/essentia/extraction/*.json
        │
        ▼
Stage 3 — Data Transformation
        scripts/es_flatten_features.py → Parquet
        dbt-core + DuckDB → fct_audio_vector_v1
        │
        ├──▶ Stage 4 — Audio Vectors
        │         scripts/qdrant_upsert_audio.py
        │         → Qdrant: sgg_audio_v1 (12-dim, cosine)
        │
        └──▶ Stage 5 — Text Embeddings
                  scripts/sgg_text_embed.py
                  Ollama: nomic-embed-text
                  → Qdrant: sgg_text_v1 (768-dim, cosine)

                            │
                            ▼
                  Stage 6 — Runtime (Streamlit Dashboard)
                            User query
                            → nomic-embed-text (query embedding)
                            → Qdrant sgg_text_v1 (retrieval)
                            → gemma3:27b (answer generation)
                            → Streamlit UI (display + album art)
```

Stage Summary

Stage	Name	Type	Script	Output
1	File Manifest	Setup	`es_select_files.py`	`data/file_lists`
2	Audio Extraction	Setup	`es_run_extractor.py`	`data/raw/essent`
3	Data Transformation	Setup	`es_flatten_features.py` + dbt	`fct_audio_vecto`
4	Audio Vectors	Setup	`qdrant_upsert_audio.py`	Qdrant `sgg_audio_`
5	Text Embeddings	Setup	`sgg_text_embed.py`	Qdrant `sgg_text_`
6	Dashboard	Runtime	`sgg_dashboard.py`	Streamlit `localhos`

Setup vs. Runtime

Setup stages (1–5) are one-time operations. They are re-run only if the source library changes, audio features are re-extracted, or a Qdrant collection is rebuilt from scratch. All outputs are persisted to disk or to Qdrant's Docker volume and survive service restarts.

Runtime (Stage 6) executes on every user interaction. The dashboard embeds the user's query, queries Qdrant, calls the LLM, and renders results — all within a single request cycle. No setup-stage scripts are involved at runtime.

Section 3: Component Deep Dives

3.1 File Manifest — scripts/es_select_files.py

Purpose: build a text file listing all extractable audio file paths in the iTunes library, for use as input to the Essentia extractor.

Inputs: iTunes library folder on the local filesystem.

Outputs: `data/file_lists/es_targets.txt` — one absolute file path per line.

Key behavior:

- Recursively walks the iTunes library directory
- Excludes DRM-protected `.m4p` files, which Essentia cannot read

- Output manifest is consumed directly by `scripts/es_run_extractor.py` in Stage 2

3.2 Audio Feature Extraction — Essentia

Purpose: extract audio features and iTunes tag metadata from each audio file, producing one JSON output per track.

Tool: `mtgupf/essentia` Docker image — Music Technology Group, Universitat Pompeu Fabra. Runs as a temporary container; not a persistent service.

Script: `scripts/es_run_extractor.py`

Inputs: `data/file_lists/es_targets.txt`

Outputs: `data/raw/essentia/extraction/*.json` — one JSON file per track, named `<basename>-<hash>.json`

Key behavior:

- Feeds files from the manifest to the Essentia Docker container sequentially
- Resume-safe — tracks completed files and skips them on restart
- Each JSON contains both computed audio features and embedded iTunes tag metadata (artist, album, title, date, genre)
- Full extraction run: ~10 hours for 9,254 tracks on local hardware

Extracted audio features:

Feature	Field	Description
Tempo	`bpm`	Beats per minute
Beat confidence	`beats_confidence`	Confidence of BPM detection
Key	`key_key`, `key_scale`	Musical key and mode (major/minor)
Key strength	`key_strength`	Confidence of key detection
Loudness	`loudness_integrated`	Integrated loudness (LUFS)
Dynamic range	`dynamic_range`	Loudness range across the track
Energy	`energy_rms`	RMS energy of the audio signal
Danceability	`danceability`	Rhythmic regularity score (0–1)

3.3 Data Transformation — dbt + DuckDB

Purpose: flatten raw Essentia JSON outputs into structured Parquet, then transform through a dbt model chain into a clean, analysis-ready feature mart.

Tools: dbt-core, DuckDB (`dbt_sgg/sgg_prod.duckdb`)

Scripts: `scripts/es_flatten_features.py`, dbt models in `dbt_sgg/models/`

Step 1 — Flatten

`scripts/es_flatten_features.py` reads all Essentia JSON files and writes structured Parquet to `data/raw/essentia/essentia_features_v1/`. One row per track; one column per feature.

Step 2 — dbt Model Chain

Model	Layer	Purpose
`ext_essentia_features`	Staging	External source — points dbt at the Parquet glob
`stg_essentia_features`	Staging	Types and cleans columns; excludes Comedy and Books & Spoken genres (40 tracks)
`im_essentia_features_unique`	Intermediate	Deduplicates by `file_hash`
`fct_audio_vector_v1`	Mart	Final feature table; produces `vector_raw` and `vector_z` (12-dim z-scored)

Z-Scoring

Z-scoring normalizes each feature to a mean of 0 and standard deviation of 1, ensuring no single feature dominates similarity calculations due to scale differences. `vector_z` is the vector loaded into Qdrant.

`fct_audio_vector_v1` emits two vector columns per track:

- `vector_raw` — raw feature values as extracted by Essentia
- `vector_z` — z-scored (standardized) version of the same 12 features

Primary key: `file_hash` — a stable hash derived from the audio file path. Carried through every downstream stage.

3.4 Vector Store — Qdrant

Purpose: store track vectors and serve similarity search queries at runtime.

Tool: Qdrant, running as a persistent Docker service (`sgg_qdrant`).

Ports: REST `6335` , gRPC `6336`

Collections:

Collection	Dims	Metric	Source	Powers
`sgg_audio_v1`	12	Cosine	`fct_audio_vector_v1.vector_z`	Audio Similarity tab
`sgg_text_v1`	768	Cosine	`nomic-embed-text` embeddings	Ask Your Library tab

Point structure: each point in Qdrant consists of:

- **ID** — `file_hash`
- **Vector** — the numeric vector used for similarity search
- **Payload** — metadata: `artist` , `album` , `title` , `date` , `genre` , `key_key` , `key_scale`

Scripts:

- `scripts/qdrant_upsert_audio.py` — loads `sgg_audio_v1` from `fct_audio_vector_v1`
- `scripts/sgg_text_embed.py` — loads `sgg_text_v1` via Ollama embeddings

Similarity search: Qdrant returns the top-k points by cosine similarity to a query vector. Cosine similarity measures the angle between two vectors — scores range from 0 (orthogonal, dissimilar) to 1 (parallel, identical). The audio similarity tab additionally filters results to exclude

tracks from the same album as the seed, over-fetching by 3× to ensure a full result set after filtering.

3.5 Text Embeddings — Ollama + nomic-embed-text

Purpose: generate 768-dimensional semantic vectors from natural language track descriptions, enabling natural language similarity search over the library.

Model: `nomic-embed-text` via Ollama (local, no cloud)

Script: `scripts/sgg_text_embed.py`

Process: for each track, a natural language sentence is constructed from its metadata and audio features:

```
{title} by {artist}, from the album {album} ({year}). Genre: {genre}.
Key: {key} {scale}, darker and more melancholic / brighter and more uplifting.
Tempo: {tempo_label} at {bpm} BPM. Energy: {energy_label}.
Groove: {danceability_label}. Dynamics: {dynamics_label}.
```

That sentence is passed to `nomic-embed-text`, which returns a 768-dimensional vector encoding its semantic meaning. The vector is upserted into `sgg_text_v1`. Descriptive labels (tempo, energy, groove, dynamics, key mood) use multiple calibrated buckets to ensure tracks are spread across the embedding space rather than clustering around generic descriptions.

Key distinction from audio vectors: audio vectors encode *what the music sounds like* based on signal analysis. Text embeddings encode *what the track is described as* based on language semantics. The two systems serve different query types and operate independently.

One-time operation: all 9,254 tracks embedded in ~3 minutes. Re-run only if track metadata changes or the collection is rebuilt.

3.6 RAG Pipeline — Retrieval-Augmented Generation

Purpose: answer natural language queries about the library with cited, grounded responses generated by a local LLM.

Models: `nomic-embed-text` (retrieval), `gemma3:27b` (generation) — both via Ollama

Script: `scripts/sgg_text_embed.py` (`rag` subcommand); also executed inline by `apps/sgg_dashboard.py` at runtime

Query execution sequence:

1. User submits a natural language query
2. Query is embedded by `nomic-embed-text` into a 768-dim vector
3. Qdrant searches `sgg_text_v1` for the top-k tracks by cosine similarity
4. Retrieved tracks are assembled into a structured prompt
5. Prompt is submitted to `gemma3:27b` via Ollama `/api/chat`
6. LLM generates a cited natural language answer, referencing only tracks it judges relevant
7. Answer and retrieved track list are returned to the dashboard for display

Grounding: the LLM receives only retrieved tracks as context — it has no access to the full library. This constrains answers to actual library content and prevents hallucination. The LLM is instructed to cite at least two tracks by artist and title and to acknowledge when retrieved context is weak.

3.7 Streamlit Dashboard — apps/sgg_dashboard.py

Purpose: provide an interactive UI integrating all SGG capabilities — RAG query, audio similarity search, and feature inspection.

Launch: `make dashboard` (starts Docker, checks Ollama, launches Streamlit)

URL: `http://localhost:8501`

Supporting module: `apps/itunes_image.py` — loads the iTunes image cache and resolves album artwork URLs by artist and album name. Cache is read once at startup; never calls the iTunes API at runtime.

Tab 1 — Ask Your Library

Executes the full RAG pipeline at query time. Retrieves a candidate pool from Qdrant (up to 5× the requested result count), then selects the final results via score-weighted random sampling — higher-scoring tracks are favoured but the outcome is not deterministic, preventing the same tracks from dominating every query. Displays the LLM-generated cited answer, a hero card (first retrieved result with a valid album image), and supporting result cards for remaining retrieved tracks.

Tab 2 — Audio Similarity Search

User selects a track via cascading Artist → Album → Track dropdowns. The dashboard retrieves the track's `vector_z` from DuckDB, queries Qdrant `sgg_audio_v1`, and displays the top-k nearest neighbors by cosine similarity. Results exclude the seed track and all tracks from the same album. Qdrant is queried with `(topk × 3) + 1` results to absorb exclusions and guarantee a full result set.

Tab 3 — Feature Inspector

User selects a track. The dashboard queries DuckDB directly for that track's raw Essentia feature values and displays them as metric tiles. No vector search or LLM involvement.

Configuration

All service endpoints are read from `.env` at startup — never hardcoded. Key variables:

Variable	Default	Purpose
`QDRANT_URL`	`http://localhost:6335`	Qdrant REST endpoint
`OLLAMA_BASE_URL_HOST`	`http://localhost:11434`	Ollama endpoint (host)
`OLLAMA_CHAT_MODEL`	`gemma3:27b`	LLM for answer generation
`OLLAMA_EMBED_MODEL`	`nomic-embed-text`	Embedding model

Section 4: Data Reference

4.1 Primary Key — file_hash

`file_hash` is a stable hash derived from the audio file path at extraction time. It serves as the primary key across all SGG data stores — Parquet files, DuckDB tables, and Qdrant collections. All joins and lookups between stages use `file_hash` as the common identifier.

4.2 dbt Mart — fct_audio_vector_v1

The terminal output of the dbt pipeline. Stored in `dbt_sgg/sgg_prod.duckdb`. This table is the source of truth for track metadata and audio features at runtime.

Column	Type	Description
`file_hash`	VARCHAR	Primary key
`artist`	VARCHAR	Artist name (iTunes tag)
`album`	VARCHAR	Album name (iTunes tag)
`title`	VARCHAR	Track title (iTunes tag)
`date`	VARCHAR	Release year (iTunes tag)
`genre`	VARCHAR	Genre (iTunes tag)
`key_key`	VARCHAR	Musical key (e.g. `A`, `F#`)
`key_scale`	VARCHAR	Mode — `major` or `minor`
`key_strength`	FLOAT	Key detection confidence (0–1)
`bpm`	FLOAT	Beats per minute
`beats_confidence`	FLOAT	BPM detection confidence (0–1)
`loudness_integrated`	FLOAT	Integrated loudness (LUFS)
`dynamic_range`	FLOAT	Loudness range
`energy_rms`	FLOAT	RMS energy
`danceability`	FLOAT	Danceability score (0–1)
`vector_raw`	FLOAT[]	12-dim raw feature array
`vector_z`	FLOAT[]	12-dim z-scored feature array

Row count: 9,254 (after deduplication and genre exclusion)

Genre exclusions: Comedy (26 tracks) and Books & Spoken (14 tracks) are filtered in `stg_essentia_features` and are not present in this table.

Known gaps: ~483 tracks missing `date`; 18 tracks missing `title`. Both are acceptable for this use case.

4.3 Qdrant Collections

sgg_audio_v1

Property	Value
Dimensions	12
Distance metric	Cosine
Point count	9,254
Vector source	`fct_audio_vector_v1.vector_z`
Point ID	`file_hash`

Payload fields per point: `file_hash`, `artist`, `album`, `title`, `date`, `genre`, `key_key`, `key_scale`

Vector dimensions (order): `danceability`, `bpm`, `loudness_integrated`, `dynamic_range`, `energy_rms`, `key_strength`, `beats_confidence`, and remaining spectral features as defined in `data/features/sgg_audio_v1_stats.json`

sgg_text_v1

Property	Value
Dimensions	768
Distance metric	Cosine
Point count	9,254
Vector source	`nomic-embed-text` embeddings of track sentences
Point ID	`file_hash`

Payload fields per point: `file_hash`, `artist`, `album`, `title`, `date`, `genre`, `key_key`, `key_scale`, `doc` (the source sentence used to generate the embedding)

4.4 iTunes Image Cache

File: `data/cache/itunes_image_cache.json`

Format: flat JSON object. Keys are `{artist}|{album}` (lowercase, stripped). Values are iTunes artwork URLs (`artworkUrl600`) or base64-encoded local JPEG strings for gap-filled entries.

Coverage: 999 unique artist/album pairs — 100% coverage (757 iTunes API hits + 52 Cover Art Archive fills + 49 manual Phish fills + 140 placeholder images).

Parquet mirror: `data/cache/itunes_image_cache.parquet` — flat table version for DuckDB inspection.

Lookup module: `apps/itunes_image.py` — `load_cache()`, `get_artwork_url()`, `resolve_hero()`

Section 5: Operations

5.1 Standard Startup

The full SGG stack is launched with a single command from the repo root:

```
make dashboard
```

This executes `scripts/start_sgg.sh`, which:

1. Starts Qdrant and Postgres via Docker Compose (`docker compose up -d`)
2. Checks whether Ollama is running — starts it if not (`ollama serve`)
3. Activates the Python venv and launches Streamlit (`streamlit run apps/sgg_dashboard.py`)

The dashboard is available at `http://localhost:8501` once Streamlit reports it is running.

5.2 Service Health Checks

Run these commands to verify each service is up before using the dashboard.

Service	Command	Expected Response
Qdrant	`curl http://localhost:6335/healthz`	`healthz check passed`
Ollama	`curl http://localhost:11434/api/tags`	JSON list of pulled models
Postgres	`docker exec sgg_postgres psql -U sgg -d sgg -c "SELECT now();"`	Current timestamp

Check Docker container status:

```
make ps
```

5.3 Stopping Services

```
make down
```

Stops the Qdrant and Postgres Docker containers. Containers are preserved — `make up` or `make dashboard` will restart them. All Qdrant vector data persists in the `qdrant_storage` Docker volume and is unaffected by stopping the service.

To stop Ollama if started manually:

```
pkill ollama
```

5.4 Container and Volume Reference

Name	Type	Purpose
`sgg_qdrant`	Container	Qdrant vector database
`sgg_postgres`	Container	Postgres (future mart target — not active in v1 pipeline)
`qdrant_storage`	Docker volume	Persists Qdrant collection data across container restarts
`pgdata`	Docker volume	Persists Postgres data

Important: `make down` runs `docker compose stop`, which stops containers without removing them. Running `docker compose down` (without the Makefile) removes containers but preserves volumes — Qdrant data survives. Running `docker compose down --volumes` removes both containers and volumes and will destroy all Qdrant collection data, requiring a full re-upsert.

5.5 Re-Running Setup Stages

Setup stages are one-time operations but may need to be re-run under specific circumstances.

Scenario	Stages to Re-Run
New tracks added to iTunes library	1 → 2 → 3 → 4 → 5
Qdrant collection corrupted or deleted	4 and/or 5 only (data in DuckDB is intact)
dbt model changes	3 → 4 → 5
Genre exclusion list updated	3 → 4 → 5
Embedding model changed	5 only (`sgg_text_v1` rebuild)

5.6 Common Issues

Dashboard opens but RAG tab returns no results

- Verify Ollama is running: `curl http://localhost:11434/api/tags`
- Verify `sgg_text_v1` is populated: `curl http://localhost:6335/collections/sgg_text_v1`
- Confirm `OLLAMA_EMBED_MODEL` and `OLLAMA_CHAT_MODEL` in `.env` match pulled model names

Audio Similarity tab returns fewer results than expected

- Expected behavior when seed track is on a heavily represented album — the same-album filter removes results and the 3× over-fetch buffer may be exhausted
- No action required

Streamlit fails to start — module not found

- Confirm the venv is active: `source venv/bin/activate`
- Confirm dependencies are installed: `pip install -r requirements.txt`

Qdrant containers missing from Docker Dashboard

- Run `make up` to recreate containers — vector data in `qdrant_storage` volume is preserved
- This occurs if `docker compose down` was run outside of the Makefile

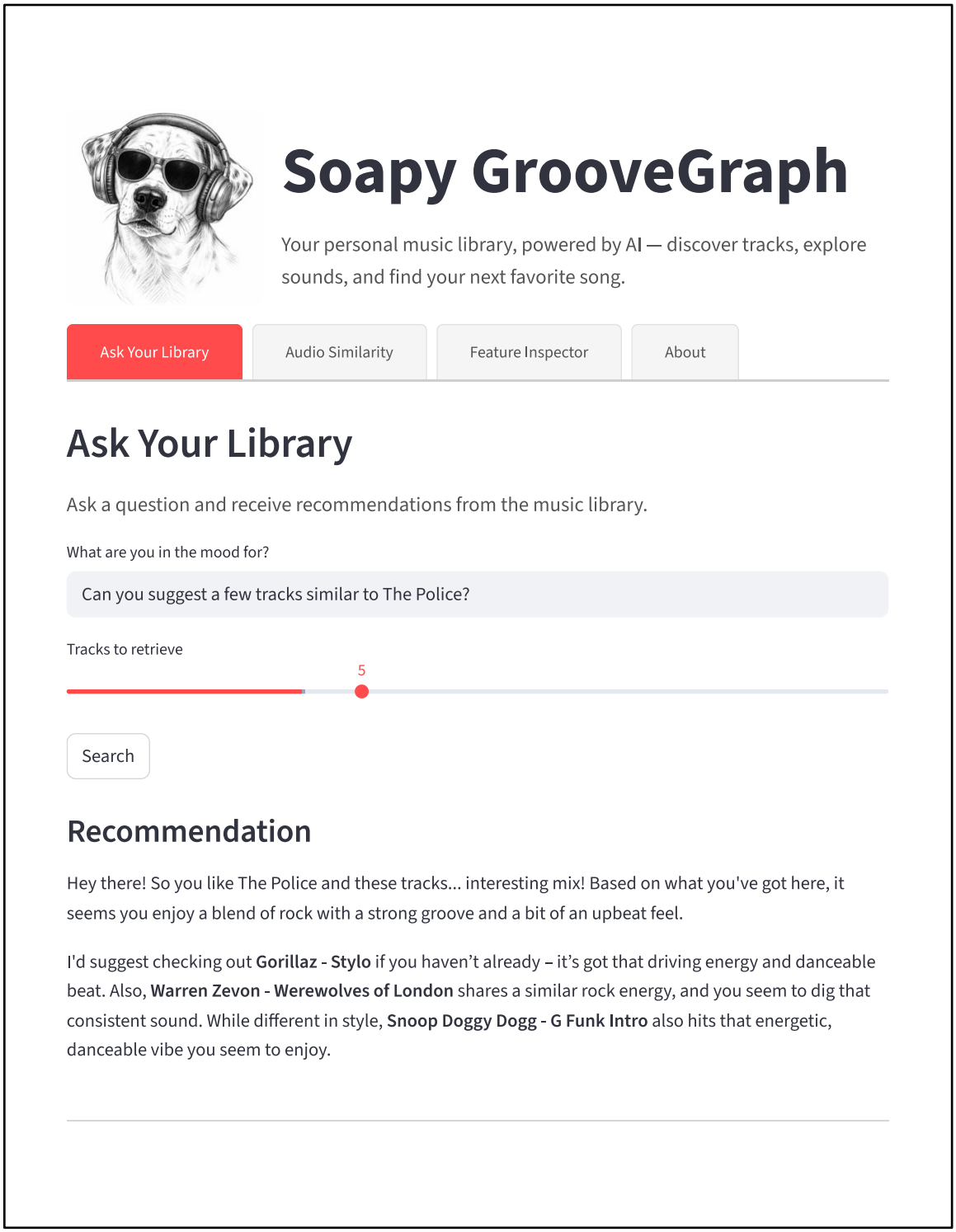
dbt run fails

- Confirm DuckDB file exists: `ls dbt_sgg/sgg_prod.duckdb`
- Run from repo root with venv active: `cd dbt_sgg && dbt run && dbt test`
- Confirm Parquet source files exist: `ls data/raw/essentia/essentia_features_v1/`

Streamlit User Interface

Four Pages in the Streamlit User Interface: 1) Ask Your Library, 2) Audio Similarity, 3) Feature Inspector, 4) About

1. Ask Your Library



2. Audio Similarity

Soapy GrooveGraph

Your personal music library, powered by AI — discover tracks, explore sounds, and find your next favorite song.

Ask Your Library | Audio Similarity | Feature Inspector | About

Audio Similarity Search

Pick a track and find the closest matches by audio features (BPM, key, energy, danceability).

Random Track

Artist

Frank Zappa

Album

Hot Rats

Track

Peaches En Regalia

Similar tracks to show

5

Find Similar Tracks

Seed: Frank Zappa — Peaches En Regalia

3. Feature Inspector

Soapy GrooveGraph

Your personal music library, powered by AI — discover tracks, explore sounds, and find your next favorite song.

Ask Your Library | Audio Similarity | Feature Inspector | About

Feature Inspector

Inspect the raw Essentia audio features for any track in the library.

Artist

A Tribe Called Quest

Album

Beats Rhymes & Life

Track

Phony Rappers

Phony Rappers

A Tribe Called Quest · *Beats Rhymes & Life* (1996)

Genre: Hip Hop/Rap

Key: F minor

Audio Features

4. About

Soapy GrooveGraph

Your personal music library, powered by AI — discover tracks, explore sounds, and find your next favorite song.

Ask Your Library | Audio Similarity | Feature Inspector | About

About Soapy GrooveGraph

What Is This?

Soapy GrooveGraph is an AI-powered music discovery tool built on top of Matt's personal iTunes library. This is not a streaming service. Nor a recommendation algorithm trained on 100 million users. Just 9,294 songs, a local AI, and the audacity to think this app should be created at all - much less for "fun."

Why Does This Exist?

Because Spotify's algorithm doesn't really know you — it knows the aggregate of everyone who listens to the same songs as you. That's not the same thing.

SGG knows *your* library. Every obscure album you imported from a CD in 2003. Every artist no one else is familiar with. Every genre mislabeled as "Other." It finds patterns in your actual collection and surfaces music you already own, but might have forgotten about.

The Objective